

1. Atividades Práticas - Componentes e Props

Nesta atividade prática, vamos trabalhar com os conceitos de sobre Componentes e Props do React.

1.1 Criando o projeto

Inicialmente, vamos criar um projeto chamado **my-components**, digitando o seguinte comando no terminal:

```
npm create vite@latest my-components -- --template react
```

Neste comando, observe que:

- **npm create vite@latest:**
 - **npm:** Refere-se ao Node Package Manager, que é o gerenciador de pacotes do Node.js.
 - **create vite@latest:** Este comando utiliza o pacote **create-vite** na sua versão mais recente (**@latest**) para inicializar um novo projeto. O **create-vite** é um gerador de projetos que configura automaticamente o ambiente de desenvolvimento com Vite.
- **my-components:**
 - Este é o nome do diretório onde o novo projeto será criado. Neste caso, o projeto será criado em uma pasta chamada **my-components**.
- **--:**
 - O **--** separa os comandos principais dos argumentos adicionais que serão passados para o **create-vite**. Isso é uma convenção para indicar que o que vem depois são opções específicas para o comando.
- **--template react:**
 - **--template:** Este argumento especifica o template que deve ser utilizado ao criar o projeto.
 - **react:** Indica que o template específico para projetos React deve ser utilizado. Isso significa que o Vite configurará automaticamente o projeto para trabalhar com React, incluindo as dependências necessárias como **react** e **react-dom**.

Este processo de configuração inicial do projeto leva alguns segundos. Ao terminar, o Vite repassa instruções para que você termine de instalar as dependências do seu projeto. Desta forma, digite o seguinte comando no terminal para **entrar no diretório** recém criado do nosso projeto, conforme quadro abaixo:

```
cd my-components
```

Em seguida, **instale as dependências** necessárias do projeto executando no terminal o seguinte comando:

```
npm install
```

Até aqui, você criou um projeto React usando o Vite e adicionou todas as dependências ao projeto. Você pode, agora, **abrir a aplicação no editor Visual Studio Code**. Para isso, dentro do diretório tutorial-react digite o seguinte comando no terminal:

```
code .
```

Após a execução deste comando, o Visual Studio Code deverá abrir com a pasta raiz do seu projeto sendo acessada.

Em seguida, você inicializará um servidor local e executará o projeto em seu navegador. Digite a seguinte linha de comando no terminal:

```
npm run dev
```

Ao executar esse script, você iniciará um servidor local de desenvolvimento, executará o código do projeto, iniciará um observador que detecta alterações no código e abrirá o projeto em um navegador web. Ele irá rodar a aplicação em modo desenvolvimento em <http://localhost:5173/>.

1.2 Desenvolvendo Componentes em React

Neste passo, vamos exercitar nossos conhecimentos desenvolvendo um conjunto de componentes em React. Primeiramente, crie um diretório chamado **componentes** na pasta **src**. Neste novo diretório, crie um arquivo chamado **PrimeiroComponente.jsx**.

Veja na imagem como ficou a estrutura do nosso projeto:

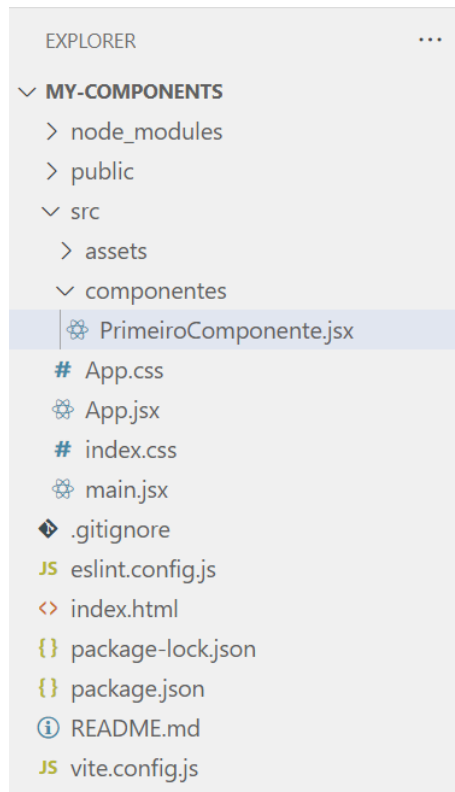


Figura 1: Estrutura do projeto my-components após a criação do arquivo PrimeiroComponente.jsx

Descrição: A imagem mostra a estrutura de diretórios e arquivos de um projeto React, criado com Vite, exibida no Visual Studio Code (VS Code), após a criação do arquivo PrimeiroComponente.jsx

No arquivo **PrimeiroComponente.jsx** vamos definir um componente de função do React, tal como o código abaixo:

```
function primeiro(){  
  return <h1>Primeiro Componente!</h1>  
}  
export default primeiro;
```

Este código define um componente funcional simples em React chamado **primeiro**. A função **primeiro** retorna um elemento JSX que renderiza um cabeçalho **<h1>** com o texto "Primeiro Componente!". O componente é então exportado como o **export** padrão do módulo, permitindo que ele seja importado e utilizado em outros arquivos do projeto React.

Note que, neste momento, qualquer alteração feita no arquivo **PrimeiroComponente.jsx** não será refletida na aplicação, ou seja, o build do React não detectará essas mudanças e a aplicação não será atualizada. Isso acontece porque o arquivo **PrimeiroComponente.jsx** não

está sendo referenciado em nenhum outro lugar da aplicação. Atualmente, o processo de build está ciente apenas do arquivo **main.jsx** e, a partir dele, navega pelos **imports** para carregar os arquivos necessários para a aplicação. Como **PrimeiroComponente.jsx** não está incluído em nenhum desses **imports**, ele não é considerado pelo build.

Logo, vamos fazer a seguinte alteração no arquivo **main.jsx**:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import PrimeiroComponente from './componentes/PrimeiroComponente';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <PrimeiroComponente />
  </StrictMode>,
)
```

Este código importa o modo estrito (**StrictMode**) e a função **createRoot** de **react** e **react-dom**, respectivamente. Ele também importa o componente **PrimeiroComponente** de um arquivo localizado no diretório **./componentes**. A função **createRoot** cria a raiz da aplicação React dentro do elemento DOM com o **id** "root". Em seguida, o método **render** é chamado para renderizar o componente **PrimeiroComponente** dentro de **<StrictMode>**, o que ajuda a detectar potenciais problemas no código e garantir que as melhores práticas sejam seguidas durante o desenvolvimento.

O resultado no navegador é mostrado na imagem abaixo:

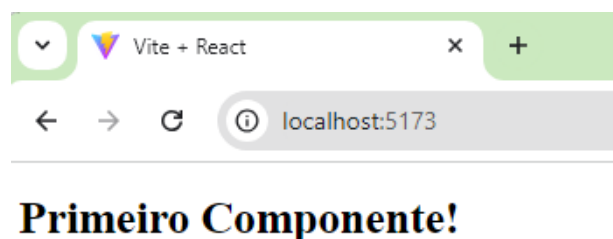


Figura 2: Resultado da renderização do componente **PrimeiroComponente** no navegador

Descrição: A imagem mostra uma aba de navegador exibindo o título "Vite + React". Na página, há um texto em negrito que diz "Primeiro Componente!". Este texto é o conteúdo renderizado de um componente React chamado **PrimeiroComponente**, que foi inserido no DOM da página.

Veja agora o resultado no DevTools do Google Chrome que temos um código HTML como resultado no navegador.



Figura 3: Estrutura HTML da página web no Google DevTools

Descrição: A imagem mostra a estrutura HTML de uma página web, exibida na aba "Elementos" do Google DevTools. O HTML é composto por uma estrutura padrão com as tags `<html>`, `<head>`, e `<body>`. No `<head>`, há scripts de módulo sendo carregados, um link para um ícone, e metadados como o charset e a configuração de viewport. No `<body>`, há uma div com o id="root", onde o conteúdo da aplicação React é montado. Dentro dessa div, está o elemento `<h1>` com o texto "Primeiro Componente!", indicando que o componente React `PrimeiroComponente` foi renderizado com sucesso na página.

Agora, vamos voltar para o nosso componente, em **PrimeiroComponente.jsx**. Considerando que ele possui apenas uma função (**function primeiro**), podemos usar uma função anônima Javascript em seu lugar. Cabe ressaltar que uma função anônima não possui um nome e, dessa forma, a única forma de utilizá-la é atribuí-la a algo (uma variável, uma propriedade em um objeto, etc).

Por exemplo, veja abaixo uma **função nomeada**:

```
// função nomeada
function common() {
  return true;
}
```

Agora, a seguir, veja uma **função anônima**:

```
// função anônima
const anonymous = function() {
  return true;
}
```

Dessa forma, podemos alterar o **PrimeiroComponente.jsx** para usar uma função anônima da seguinte forma:

```
export default function () {
  return <h1>Primeiro Componente!</h1>
}
```

OBS: Talvez o ESLint mostre algumas mensagens de warning.

Ainda, podemos alterar a forma que criamos este componente de função usando uma **arrow function**. Cabe lembrar que as arrow functions (ou funções de seta) foram introduzidas no ES6 e permitem escrever funções com uma sintaxe mais curta.

```
export default () => <h1>Primeiro Componente!</h1>
```

OBS: Talvez o ESLint mostre algumas mensagens de warning.

1.3 Trabalhando com props

Como vimos anteriormente, **props** são as entradas dos componentes React, representando dados que são passados de um componente pai para um componente filho. No entanto, é importante lembrar que **props** são somente leitura, ou seja, não devem ser modificados diretamente.

Portanto, você não deve tentar alterá-los da seguinte maneira:

```
// Errado!  
props.numero = 42;
```

Vejamos um exemplo do uso de props ao alterar o nosso componente **PrimeiroComponente.jsx**:

```
function primeiro(props){  
    return <h1>Primeiro Componente! {props.valor}</h1>  
}  
export default primeiro;
```

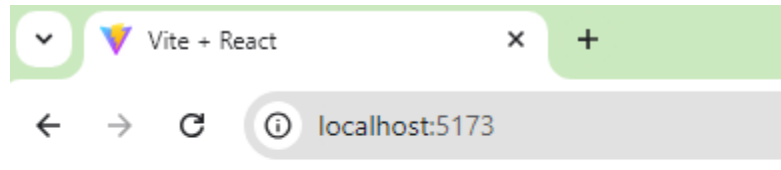
Observe que o componente de função **primeiro** está retornando código JSX. Dentro desse contexto, usamos chaves { } para inserir um valor JavaScript no JSX. Sem as chaves, o React interpretaria o conteúdo como texto simples. Além disso, o nome do parâmetro não precisa ser necessariamente **props**; você tem a liberdade de escolher outro nome, mas **props** é o nome comumente utilizado por convenção.

Agora, para testar a passagem de parâmetros para este componente, precisamos modificar **main.jsx** para ficar assim:

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import PrimeiroComponente from './componentes/PrimeiroComponente';  
  
createRoot(document.getElementById('root')).render(  
    <StrictMode>  
        <PrimeiroComponente valor="Bom dia"/>  
    </StrictMode>,  
)
```

Neste código, o método **render** é chamado para renderizar o componente **PrimeiroComponente**, passando um **prop** chamado **valor** com o texto "Bom dia".

Como resultado, no navegador temos:



Primeiro Componente! Bom dia

Figura 4: Resultado da renderização do componente PrimeiroComponente no navegador com o uso de props

Descrição: A imagem mostra uma aba de navegador exibindo o título "Vite + React". Na página, há um texto em negrito que diz "Primeiro Componente! Bom dia". Isso significa que o componente React renderizado inclui tanto a mensagem "Primeiro Componente!" quanto "Bom dia", ambas sendo exibidas juntas como parte do conteúdo da aplicação.

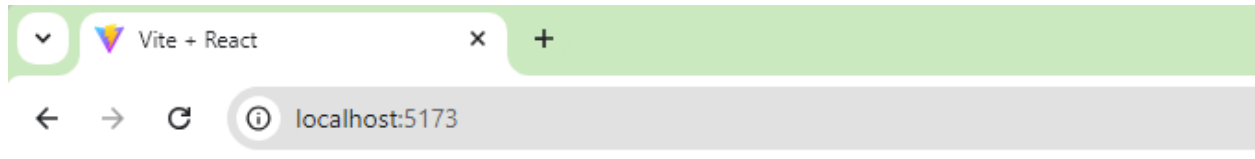
Podemos, ainda, passar mais de um parâmetro. Este segundo parâmetro vai passar o resultado de uma função Javascript. Veja o arquivo **PrimeiroComponente.jsx**:

```
function primeiro(props){  
    return <h1>Primeiro Componente! {props.valor} - {props.valor2} </h1>  
}  
export default primeiro;
```

No **main.jsx**, alteramos o código assim:

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import PrimeiroComponente from './componentes/PrimeiroComponente';  
  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <PrimeiroComponente valor="Bom dia" valor2={Math.random()} />  
  </StrictMode>,  
)
```


O resultado no navegador é o seguinte:



Primeiro Componente! Bom dia - 0.04379146548633095

Figura 5: Resultado da renderização do componente PrimeiroComponente no navegador com o uso de duas props

Descrição: A imagem mostra uma aba de navegador com o título "Vite + React". Na página, é exibido um texto em negrito que diz "Primeiro Componente! Bom dia - 0.04379146548633095". Isso sugere que o componente React renderizado mostra a mensagem "Primeiro Componente! Bom dia" seguida por um número decimal aleatório. O número foi gerado dinamicamente, como parte de uma expressão JavaScript dentro do componente.

1.3.1 Uso de props.children

Em React, **props.children** é uma propriedade especial que permite que você passe elementos ou componentes como filhos de um componente. Quando você define um componente, **props.children** representa qualquer conteúdo que seja passado entre as **tags de abertura e fechamento** do componente ao usá-lo. Isso permite que você crie componentes "contêiner", onde você pode definir a estrutura geral do componente e deixar o conteúdo específico para ser definido ao usar o componente.

Por exemplo, crie na pasta componentes um arquivo **SegundoComponente.jsx** com o seguinte código:

```
function SegundoComponente(props) {  
  return <p>{props.children}</p>;  
}  
export default SegundoComponente;
```

Em seguida, altere o arquivo **main.jsx** para chamar este componente chamado **SegundoComponente**:

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import SegundoComponente from './componentes/SegundoComponente';
```

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <SegundoComponente>Seja bem vindo!!!</SegundoComponente>  
  </StrictMode>,  
)
```

Veja o resultado no navegador:

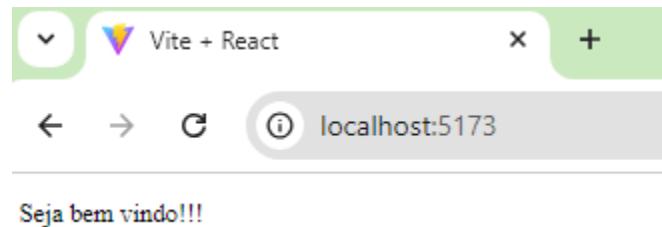


Figura 6: Resultado da renderização do componente SegundoComponente no navegador

Descrição: A imagem mostra uma aba de navegador com o título "Vite + React" e o endereço local `http://localhost:5173`, indicando que uma aplicação React está sendo executada localmente usando Vite. No corpo da página, há um texto simples que diz "Seja bem vindo!!!".

1.3.2 Múltiplos Componentes em um arquivo

Embora o React permita que você defina múltiplos componentes em um único arquivo, a prática comum é criar um arquivo separado para cada componente. Isso facilita a organização e a manutenção do código. Vamos agora criar um arquivo chamado **DoisComponentes.jsx** na pasta **componentes**.

Em seguida, digite o seguinte código para **DoisComponentes.jsx**:

```
const CompA = props =>  
  <h1>Primeiro diz: {props.valor}</h1>  
const CompB = props =>  
  <h1>Segundo diz: {props.valor}</h1>  
export {CompA, CompB};
```

Observe que temos dois componentes (**CompA** e **CompB**) definidos neste arquivo e que não temos nenhum componente exportado por **default** (isso vai gerar uma pequena mudança no momento de fazer o **import**).

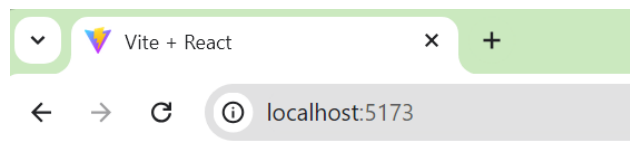
Em **main.jsx** vamos importar estes componentes e usá-los em nosso aplicativo:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { CompA, CompB } from './componentes/DoisComponentes';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <div>
      <CompA valor="Olá, eu sou A!" />
      <CompB valor="B está na área!" />
    </div>
  </StrictMode>,
)
```

Este código importa o modo estrito (**StrictMode**) e a função **createRoot** de **react** e **react-dom**, além dos componentes **CompA** e **CompB** do arquivo **DoisComponentes.jsx**. O **createRoot** cria a raiz da aplicação React dentro do elemento DOM com o **id="root"**. Em seguida, o método **render** é usado para renderizar os componentes **CompA** e **CompB** dentro de uma **<div>**. Cada componente recebe uma **prop** chamada **valor** com diferentes textos, que são renderizados na tela dentro do **StrictMode**, garantindo que as melhores práticas sejam seguidas durante o desenvolvimento.

Veja o resultado no navegador:



Primeiro diz: Olá, eu sou A!

Segundo diz: B está na área!

Figura 7: Resultado da renderização do componente DoisComponentes no navegador

Descrição: A imagem mostra uma aba de navegador com o título "Vite + React" e o endereço local <http://localhost:5173>, indicando que uma aplicação React está sendo executada localmente usando Vite.

No corpo da página, há dois textos em negrito: "Primeiro diz: Olá, eu sou A!" e "Segundo diz: B está na área!" Esses textos foram provavelmente gerados por dois componentes React distintos, **CompA** e **CompB**, que receberam as mensagens "Olá, eu sou A!" e "B está na área!" como propriedades (props).

Podemos definir um apelido para os componentes. Por exemplo, podemos dar um apelido para o componente B, desta forma:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { CompA, CompB as B } from './componentes/DoisComponentes';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <div>
      <CompA valor="Olá, eu sou A!" />
      <B valor="B está na área!" />
    </div>
  </StrictMode>,
)
```

1.3.3 Retornando Múltiplos Elementos em um Componente

Veremos agora uma situação quando um componente precisa retornar múltiplos elementos adjacentes, ou seja, aqueles elementos que estão um ao lado do outro, geralmente dentro de um elemento maior, como uma `<div>` ou uma `<section>`.

Para exemplificar esta situação, vamos criar um arquivo chamado **MultiElementos.jsx** em **componentes** e adicionar o seguinte código:

```
function MultiElementos(){
  return (
    <h1>Parte 1</h1>
    <h2>Parte 2</h2>
  )
}
export default MultiElementos;
```

OBS: Observe que o VS Code já acusa um problema.

Em seguida, no arquivo **main.jsx** vamos utilizar este componente da seguinte forma:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import MultiElementos from './componentes/MultiElementos';
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <MultiElementos />
  </StrictMode>,
)
```

Entretanto, podemos verificar que o compilador e o navegador acusam o seguinte erro:

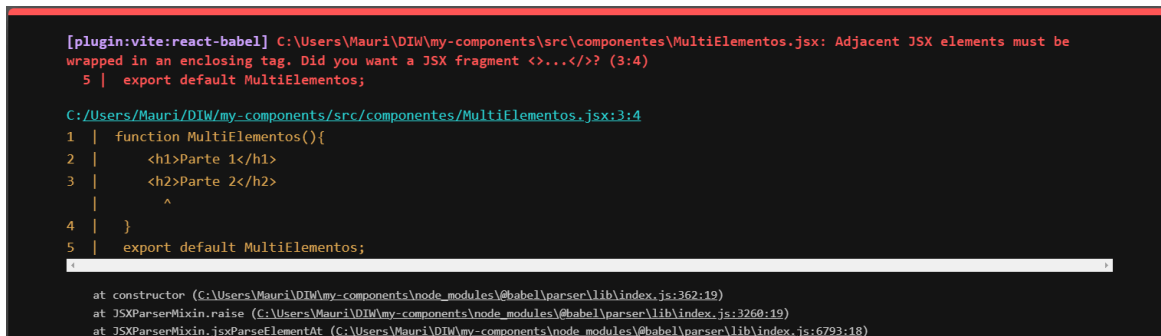


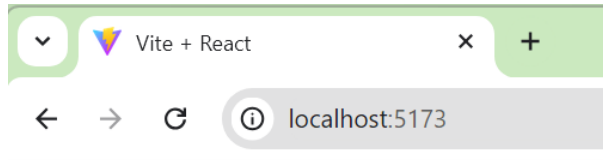
Figura 8: Mensagem de erro no navegador ao renderizar o componente MultiElementos

Descrição: A imagem mostra uma mensagem de erro no terminal ou console do Visual Studio Code, gerada ao tentar executar um arquivo React chamado MultiElementos.jsx. O erro indica que há elementos JSX adjacentes que precisam estar encapsulados dentro de uma única tag pai.

No código, há dois elementos JSX adjacentes (`<h1>Parte 1</h1>` e `<h2>Parte 2</h2>`) que não estão dentro de uma tag contêiner, como um `<div>`. Por isso, o compilador React está gerando um erro. O erro sugere que você deve envolver esses elementos em uma tag única para que o código seja válido. Para resolver esse problema, podemos envolver o retorno dentro de uma tag `<div>` no arquivo **MultiElementos.jsx**:

```
function MultiElementos(){
  return (
    <div>
      <h1>Parte 1</h1>
      <h2>Parte 2</h2>
    </div>
  )
}
export default MultiElementos;
```

Veja o resultado no navegador:



Parte 1

Parte 2

Figura 9: Mensagem de erro no navegador ao renderizar o componente MultiElementos

Descrição: A imagem mostra uma aba de navegador com o título "Vite + React". Na página, são exibidos dois textos em negrito: "Parte 1" e "Parte 2". Esses textos são exibidos em linhas separadas, sugerindo que foram gerados por um componente React que renderizou dois elementos, provavelmente um `<h1>` para "Parte 1" e um `<h2>` para "Parte 2".

Podemos inspecionar esse resultado utilizando o DevTools do Google Chrome, acessando a aba 'Elementos' (ou 'Elements'), para visualizar o código HTML gerado:



Figura 10: Estrutura HTML da página web no Google DevTools

Descrição: A imagem mostra a estrutura HTML de uma página web, exibida na aba "Elementos" do Google DevTools. O HTML é composto por uma estrutura padrão com as tags `<html>`, `<head>`, e `<body>`. No corpo da página (`<body>`), há uma div com o id="root", que serve como o contêiner principal para a aplicação React. Dentro dessa div, há outra div que contém dois elementos: Um cabeçalho `<h1>` com o texto "Parte 1" e um cabeçalho `<h2>` com o texto "Parte 2".

Outra solução seria utilizar um fragmento JSX (`<>...</>`), que permite encapsular múltiplos elementos adjacentes com uma 'tag fantasma', ou seja, uma tag que não será renderizada no HTML final. Veja como isso pode ser aplicado no arquivo **MultiElementos.jsx**:

```
function MultiElementos(){
  return (
    <>
      <h1>Parte 1</h1>
      <h2>Parte 2</h2>
    </>
  )
}
export default MultiElementos;
```

Note que o resultado final no navegador permanece o mesmo. No entanto, ao inspecionar o resultado no DevTools do Google Chrome, você perceberá que não foi necessário 'poluir' nosso código com uma `<div>` desnecessária.

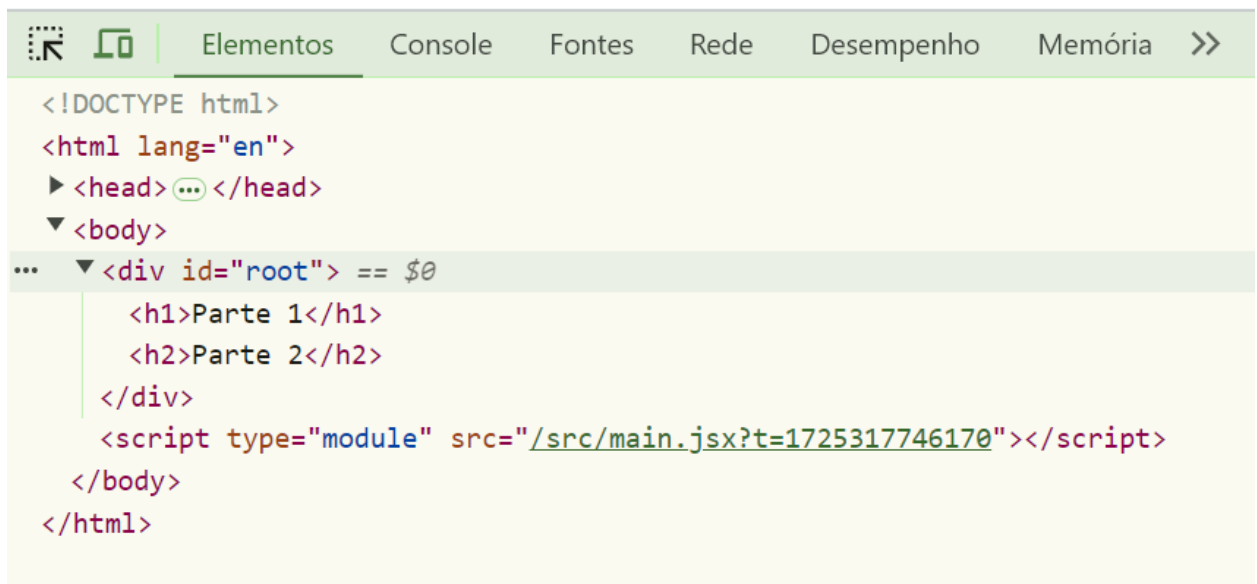


Figura 11: Estrutura HTML da página web no Google DevTools sem a `<div>` adicional

Descrição: A imagem mostra a estrutura HTML de uma página web, exibida na aba "Elementos" do Google DevTools. No corpo da página (`<body>`), há uma `div` com o `id="root"`, que serve como o contêiner principal para a aplicação React. Dentro dessa `div`, contém dois elementos: Um cabeçalho `<h1>` com o texto "Parte 1" e um cabeçalho `<h2>` com o texto "Parte 2".